Computer Vision — Revision

Question 1: Gaussian & Lorentzian Kernels

The Lorentzian function $\dfrac{1}{x^2+1}$ can approximate the Gaussian function $\dfrac{1}{\sqrt{2\pi\sigma^2}}\,e^{-x^2/2\sigma^2}$.

1. What will be the 1×3 and 1×5 kernels using the Gaussian kernel?
2. What will be the 1×3 and 1×5 kernels using the Lorentzian kernel?
3. What can be a 2D 3×3 or 5×5 kernel using the Lorentzian function? Assuming separability can be applied.
4. What would be the equivalent σ of the Gaussian kernel that produces the same filter as the Lorentzian function if the filter size is 1×3? Try some σ values if the exact value cannot be found and report the closest one.

Answers

1. Gaussian Kernel The Gaussian filter is $\dfrac{1}{\sqrt{2\pi\sigma^2}}\,e^{-x^2/2\sigma^2}$.

1×3 kernel: Evaluating at $x \in \{-1, 0, 1\}$, the unnormalized weights are $e^{-1/2\sigma^2},\ 1,\ e^{-1/2\sigma^2}$. The normalisation factor is

$$W_p = \frac{1}{\sqrt{2\pi\sigma^2}}\left(1 + 2e^{-1/2\sigma^2}\right) = \frac{2e^{-1/2\sigma^2}+1}{\sqrt{2\pi\sigma^2}}$$

so the normalised kernel is:

$$\frac{1}{2e^{-1/2\sigma^2}+1}\begin{bmatrix} e^{-1/2\sigma^2} & 1 & e^{-1/2\sigma^2} \end{bmatrix}$$

1×5 kernel: Evaluating at $x \in \{-2, -1, 0, 1, 2\}$:

$$\frac{1}{2e^{-2/\sigma^2}+2e^{-1/2\sigma^2}+1}\begin{bmatrix} e^{-2/\sigma^2} & e^{-1/2\sigma^2} & 1 & e^{-1/2\sigma^2} & e^{-2/\sigma^2} \end{bmatrix}$$

2. Lorentzian Kernel The Lorentzian function is $\dfrac{1}{x^2+1}$.

1×3 kernel (values at $x \in \{-1, 0, 1\}$: $\frac{1}{2}, 1, \frac{1}{2}$, sum $= 2$):

$$\frac{1}{2}\begin{bmatrix} \frac{1}{2} & 1 & \frac{1}{2} \end{bmatrix}$$

1×5 kernel (values at $x \in \{-2,-1,0,1,2\}$: $\frac{1}{5}, \frac{1}{2}, 1, \frac{1}{2}, \frac{1}{5}$, sum $= \frac{12}{5}$):

$$\frac{5}{12}\begin{bmatrix}\frac{1}{5} & \frac{1}{2} & 1 & \frac{1}{2} & \frac{1}{5}\end{bmatrix}$$

3. 2D Lorentzian Kernels (separable) **3×3 kernel** (outer product of two 1×3 kernels):

$$\frac{1}{4}\begin{bmatrix}\frac{1}{4} & \frac{1}{2} & \frac{1}{4} \\ \frac{1}{2} & 1 & \frac{1}{2} \\ \frac{1}{4} & \frac{1}{2} & \frac{1}{4}\end{bmatrix}$$

5×5 kernel (outer product of two 1×5 kernels, normalisation $= \frac{144}{25}$):

$$\frac{25}{144}\begin{bmatrix}\frac{1}{25} & \frac{1}{10} & \frac{1}{5} & \frac{1}{10} & \frac{1}{25} \\ \frac{1}{10} & \frac{1}{4} & \frac{1}{2} & \frac{1}{4} & \frac{1}{10} \\ \frac{1}{5} & \frac{1}{2} & 1 & \frac{1}{2} & \frac{1}{5} \\ \frac{1}{10} & \frac{1}{4} & \frac{1}{2} & \frac{1}{4} & \frac{1}{10} \\ \frac{1}{25} & \frac{1}{10} & \frac{1}{5} & \frac{1}{10} & \frac{1}{25}\end{bmatrix}$$

4. Equivalent σ for Gaussian $\approx$ Lorentzian (1×3) The Lorentzian 1×3 kernel is proportional to $[\frac{1}{2}, 1, \frac{1}{2}]$. The Gaussian 1×3 kernel is proportional to $[e^{-1/2\sigma^2}, 1, e^{-1/2\sigma^2}]$.

We require:

$$\frac{e^{-1/2\sigma^2}}{2e^{-1/2\sigma^2}+1} = \frac{1/2}{2} = \frac{1}{4}$$

$$\implies 2e^{-1/2\sigma^2} + 1 = 2 \implies e^{-1/2\sigma^2} = \frac{1}{2}$$

$$-\frac{1}{2\sigma^2} = \ln(0.5) \implies \sigma = \sqrt{\frac{1}{2\ln 2}} \approx 0.849 \approx 0.85$$

Verification: $e^{-1/(2\times 0.85^2)} \approx \frac{1}{4}$ of the normalised kernel. ✓

Question 2: Filtering

Given the 5×5 image window:

$$F(x,y) = \begin{bmatrix}2 & 3 & 4 & 5 & 6 \\ 3 & 4 & 5 & 6 & 8 \\ 4 & 5 & 6 & 8 & 5 \\ 5 & 7 & 8 & 9 & 3 \\ 9 & 10 & 9 & 4 & 3\end{bmatrix}$$

Apply the following:

A. Non-Maximum Suppression within a 3×3 window for every pixel.

B. Median filter 3×3 for the centre pixel.

C. Sobel Horizontal filter for the centre pixel (write the filter, then compute).

D. Laplacian of Gaussian filter 5×5 with $\sigma = 1/\sqrt{2}$ for the centre pixel.

Answers

A. Non-Maximum Suppression (3×3) For each pixel (r, c), keep its value if and only if it is the maximum in its 3×3 neighbourhood; otherwise set to 0. Border pixels use zero-padding.

Row 1 (top row, zero-padded above):

- $(1,1) = 2$: window max $= 4$. $2 < 4 \Rightarrow \mathbf{0}$
- $(1,2) = 3$: window max $= 5$. $3 < 5 \Rightarrow \mathbf{0}$
- $(1,3) = 4$: window max $= 6$. $4 < 6 \Rightarrow \mathbf{0}$
- $(1,4) = 5$: window max $= 8$. $5 < 8 \Rightarrow \mathbf{0}$
- $(1,5) = 6$: window $\{0,0,0,\ 5,6,0,\ 6,8,0\}$, max $= 8$. $6 < 8 \Rightarrow \mathbf{0}$

Row 2:

- $(2,1) = 3$: window $\{0,2,3,\ 0,3,4,\ 0,4,5\}$, max $= 5$. $3 < 5 \Rightarrow \mathbf{0}$
- $(2,2) = 4$: window $\{2,3,4,\ 3,4,5,\ 4,5,6\}$, max $= 6$. $4 < 6 \Rightarrow \mathbf{0}$
- $(2,3) = 5$: window $\{3,4,5,\ 4,5,6,\ 5,6,8\}$, max $= 8$. $5 < 8 \Rightarrow \mathbf{0}$
- $(2,4) = 6$: window $\{4,5,6,\ 5,6,8,\ 6,8,5\}$, max $= 8$. $6 < 8 \Rightarrow \mathbf{0}$
- $(2,5) = 8$: window $\{5,6,0,\ 6,8,0,\ 8,5,0\}$, max $= 8$. $8 = 8 \Rightarrow$ **KEEP: 8**

Row 3:

- $(3,1) = 4$: window $\{0,3,4,\ 0,4,5,\ 0,5,7\}$, max $= 7$. $4 < 7 \Rightarrow \mathbf{0}$
- $(3,2) = 5$: window $\{3,4,5,\ 4,5,6,\ 5,7,8\}$, max $= 8$. $5 < 8 \Rightarrow \mathbf{0}$
- $(3,3) = 6$: window $\{4,5,6,\ 5,6,8,\ 7,8,9\}$, max $= 9$. $6 < 9 \Rightarrow \mathbf{0}$
- $(3,4) = 8$: window $\{5,6,8,\ 6,8,5,\ 8,9,3\}$, max $= 9$. $8 < 9 \Rightarrow \mathbf{0}$
- $(3,5) = 5$: window $\{6,8,0,\ 8,5,0,\ 9,3,0\}$, max $= 9$. $5 < 9 \Rightarrow \mathbf{0}$

Row 4:

- $(4,1) = 5$: window $\{0,4,5,\ 0,5,7,\ 0,9,10\}$, max $= 10$. $5 < 10 \Rightarrow \mathbf{0}$
- $(4,2) = 7$: window $\{4,5,6,\ 5,7,8,\ 9,10,9\}$, max $= 10$. $7 < 10 \Rightarrow \mathbf{0}$
- $(4,3) = 8$: window $\{5,6,8,\ 7,8,9,\ 10,9,4\}$, max $= 10$. $8 < 10 \Rightarrow \mathbf{0}$
- $(4,4) = 9$: window $\{6,8,5,\ 8,9,3,\ 9,4,3\}$, max $= 9$. $9 = 9 \Rightarrow$ **KEEP: 9**
- $(4,5) = 3$: window $\{8,5,0,\ 9,3,0,\ 4,3,0\}$, max $= 9$. $3 < 9 \Rightarrow \mathbf{0}$

Row 5 (bottom row, zero-padded below):

- $(5,1) = 9$: window $\{0,5,7,\ 0,9,10,\ 0,0,0\}$, max $= 10$. $9 < 10 \Rightarrow \mathbf{0}$
- $(5,2) = 10$: window $\{5,7,8,\ 9,10,9,\ 0,0,0\}$, max $= 10$. $10 = 10 \Rightarrow$ **KEEP: 10**
- $(5,3) = 9$: window $\{7,8,9,\ 10,9,4,\ 0,0,0\}$, max $= 10$. $9 < 10 \Rightarrow \mathbf{0}$
- $(5,4) = 4$: window $\{8,9,3,\ 9,4,3,\ 0,0,0\}$, max $= 9$. $4 < 9 \Rightarrow \mathbf{0}$
- $(5,5) = 3$: window $\{9,3,0,\ 4,3,0,\ 0,0,0\}$, max $= 9$. $3 < 9 \Rightarrow \mathbf{0}$

Final NMS Result (three local maxima survive):

$$\mathrm{NMS}(F) = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 8 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 9 & 0 \\ 0 & 10 & 0 & 0 & 0 \end{bmatrix}$$

B. Median Filter (3×3, centre pixel) The centre pixel is at position $(3, 3)$ (1-indexed), value $= 6$. The 3×3 neighbourhood centred there is:

$$\begin{bmatrix} 4 & 5 & 6 \\ 5 & 6 & 8 \\ 7 & 8 & 9 \end{bmatrix}$$

Sorted: $4, 5, 5, 6, \mathbf{6}, 7, 8, 8, 9$. Median (5th value) $= \mathbf{6}$.

After applying the filter nothing will change.

C. Sobel Horizontal Filter (centre pixel) The Sobel horizontal kernel is:

$$S_x = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

Applying to the 3×3 neighbourhood of the centre:

$$(1)(4)+(0)(5)+(-1)(6)+(2)(5)+(0)(6)+(-2)(8)+(1)(7)+(0)(8)+(-1)(9) = 4-6+10-16+7-9 = \mathbf{-10}$$

D. Laplacian of Gaussian (5×5, $\sigma = 1/\sqrt{2}$) Step 1 — Derive the LoG from the Gaussian.

Start with the 2D Gaussian:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)/2\sigma^2}$$

First partial with respect to x:

$$\frac{\partial G}{\partial x} = \frac{1}{2\pi\sigma^2} \cdot \left(-\frac{x}{\sigma^2}\right) e^{-(x^2+y^2)/2\sigma^2}$$

Second partial with respect to x:

$$\frac{\partial^2 G}{\partial x^2} = \frac{1}{2\pi\sigma^2} \left(\frac{x^2}{\sigma^4} - \frac{1}{\sigma^2}\right) e^{-(x^2+y^2)/2\sigma^2}$$

By symmetry in y:

$$\frac{\partial^2 G}{\partial y^2} = \frac{1}{2\pi\sigma^2} \left(\frac{y^2}{\sigma^4} - \frac{1}{\sigma^2}\right) e^{-(x^2+y^2)/2\sigma^2}$$

Summing (the Laplacian):

$$\boxed{\nabla^2 G = \frac{1}{2\pi\sigma^4} \cdot \frac{x^2 + y^2 - 2\sigma^2}{\sigma^2} e^{-(x^2+y^2)/2\sigma^2}}$$

Which we write compactly as:

$$h(x, y) = \frac{1}{2\pi\sigma^2} \cdot \frac{x^2 + y^2 - 2\sigma^2}{\sigma^4} e^{-(x^2+y^2)/2\sigma^2}$$

Using $\sigma^2 = 0.5$, the filter weights evaluated at different offsets (x, y) yield approximately:

$$\frac{1}{2\pi(0.5)} \begin{bmatrix} 0.135 & 0.0716 & 0 & 0.0716 & 0.135 \\ 0.0716 & -0.3 & -0.58 & -0.3 & 0.0716 \\ 0 & -0.58 & -1 & -0.58 & 0 \\ 0.0716 & -0.3 & -0.58 & -0.3 & 0.0716 \\ 0.135 & 0.0716 & 0 & 0.0716 & 0.135 \end{bmatrix}$$

Final result calculation:

$$\begin{aligned} \text{Result} = \frac{1}{\pi} \times [&0.135(2+6+3+9) + 0.0716(3+5+3+8+4+10+3+5) \\ &- 0.3(4+6+7+9) - 0.58(5+5+8+8) - 1(6)] = -1.849 \end{aligned}$$

Question 3: Bilateral Filter & Corner Detection

Given the 5×5 image patch:

$$F(x, y) = \begin{bmatrix} 10 & 10 & 20 & 30 & 30 \\ 10 & 10 & 20 & 30 & 30 \\ 30 & 30 & 50 & 70 & 70 \\ 10 & 10 & 20 & 30 & 30 \\ 10 & 10 & 20 & 30 & 30 \end{bmatrix}$$

with $\sigma_s = 1$, $\sigma_r = 20$. Centre pixel $F_c = 50$ at position $(3, 3)$.

1. Construct the bilateral filter for the central pixel.
2. Compute the x- and y-derivatives using Sobel filters for the centre pixel.
3. Compute the cornerness score for the middle pixel assuming a 3×3 neighbourhood.

Answers

1. Bilateral Filter — Detailed Construction $\sigma_s = 1, \quad \sigma_r = 20$

Spatial distance matrix E^2 and intensity difference matrix D:

$$E^2 = \begin{bmatrix} 2 & 1 & 2 \\ 1 & 0 & 1 \\ 2 & 1 & 2 \end{bmatrix}, \qquad D = \begin{bmatrix} 40 & 30 & 20 \\ 20 & 0 & 20 \\ 40 & 30 & 20 \end{bmatrix}$$

Spatial filter component $\left(e^{-E^2/(2\sigma_s^2)}\right)$:

$$\begin{bmatrix} e^{-1} & e^{-0.5} & e^{-1} \\ e^{-0.5} & 1 & e^{-0.5} \\ e^{-1} & e^{-0.5} & e^{-1} \end{bmatrix}$$

Range filter component $\left(e^{-D^2/(2\sigma_r^2)}\right)$:

$$\begin{bmatrix} e^{-2} & e^{-1.125} & e^{-0.5} \\ e^{-0.5} & 1 & e^{-0.5} \\ e^{-2} & e^{-1.125} & e^{-0.5} \end{bmatrix}$$

Bilateral filter (unnormalized):

$$\begin{bmatrix} 0.05 & 0.197 & 0.22 \\ 0.368 & 1 & 0.368 \\ 0.05 & 0.197 & 0.22 \end{bmatrix}$$

Sum of weights:

$$W = 2(0.05) + 2(0.197) + 2(0.22) + 2(0.368) + 1 = 2.67.$$

2. Compute the x- and y-derivatives using Sobel filters for the centre pixel of $F(x, y)$.

$$S_x = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}, \qquad S_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

For the middle pixel:

$$I_x = 10 - 30 + 2(30) - 2(70) + 10 - 30 = -120$$
$$I_y = 10 + 10 + 2(20) - 2(20) + 30 - 30 = 0$$

3. Cornerness score for the middle pixel (3×3 neighbourhood). Using the simple 1×3 / 3×1 difference kernels:

$$S_x = \begin{bmatrix} 1 & 0 & -1 \end{bmatrix}, \qquad S_y = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}$$

Derivative matrices I_x and I_y for the 3×3 neighbourhood:

$$I_x = \begin{bmatrix} -10 & -20 & -10 \\ -20 & -40 & -20 \\ -10 & -20 & -10 \end{bmatrix}, \qquad I_y = \begin{bmatrix} -20 & -30 & -40 \\ 0 & 0 & 0 \\ 20 & 30 & 40 \end{bmatrix}$$

Step 1 — Mean normalisation.

$$\mu_{I_x} = \frac{-10 - 20 - 10 - 20 - 40 - 20 - 10 - 20 - 10}{9} = -\frac{160}{9} \approx -17.78, \qquad \mu_{I_y} = 0$$

Subtract the means to obtain $\widetilde{I}_x = I_x - \mu_{I_x}$ and $\widetilde{I}_y = I_y - \mu_{I_y}$:

$$\widetilde{I}_x \approx \begin{bmatrix} 7.78 & -2.22 & 7.78 \\ -2.22 & -22.22 & -2.22 \\ 7.78 & -2.22 & 7.78 \end{bmatrix}, \qquad \widetilde{I}_y = \begin{bmatrix} -20 & -30 & -40 \\ 0 & 0 & 0 \\ 20 & 30 & 40 \end{bmatrix}$$

Step 2 — Covariance matrix.

Using the shortcut $\sum \widetilde{I}^2 = \sum I^2 - n\mu^2$:

$$\sum \widetilde{I}_x^2 = 3600 - 9\left(-\tfrac{160}{9}\right)^2 = 3600 - \tfrac{25600}{9} = \tfrac{6800}{9} \approx 755.56$$

$$\sum \widetilde{I}_y^2 = 5800 - 9(0)^2 = 5800$$

$$\sum \widetilde{I}_x \widetilde{I}_y = 0 \quad \text{(by symmetry)}$$

$$M = \begin{bmatrix} \sum \widetilde{I}_x^2 & \sum \widetilde{I}_x \widetilde{I}_y \\ \sum \widetilde{I}_x \widetilde{I}_y & \sum \widetilde{I}_y^2 \end{bmatrix} = \begin{bmatrix} 755.56 & 0 \\ 0 & 5800 \end{bmatrix}$$

Step 3 — Cornerness score.

$$\text{Cornerness} = \det(M) - k\big[\operatorname{tr}(M)\big]^2 = (5800 \times 755.56) - k\,(755.56 + 5800)^2$$

Eigenvalues: $\lambda_1 = 5800$, $\lambda_2 = 755.56$. Since $\lambda_1 \gg \lambda_2$, there is likely an **edge**; however, because λ_2 is still fairly large, a **corner** is also plausible.

Alternative Solution

2. Sobel Derivatives Step 1 — Kernels.

$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \qquad S_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Step 2 — 3×3 neighbourhood of centre.

$$P = \begin{bmatrix} 10 & 20 & 30 \\ 30 & 50 & 70 \\ 10 & 20 & 30 \end{bmatrix}$$

Step 3 — Compute I_x at the centre.

$$\begin{aligned} I_x &= (-1)(10) + (0)(20) + (1)(30) \\ &\quad + (-2)(30) + (0)(50) + (2)(70) \\ &\quad + (-1)(10) + (0)(20) + (1)(30) \\ &= (-10 + 0 + 30) + (-60 + 0 + 140) + (-10 + 0 + 30) \\ &= 20 + 80 + 20 = \mathbf{120} \end{aligned}$$

Step 4 — Compute I_y at the centre.

$$\begin{aligned} I_y &= (-1)(10) + (-2)(20) + (-1)(30) \\ &\quad + (0)(30) + (0)(50) + (0)(70) \\ &\quad + (1)(10) + (2)(20) + (1)(30) \\ &= (-10 - 40 - 30) + 0 + (10 + 40 + 30) \\ &= -80 + 0 + 80 = \mathbf{0} \end{aligned}$$

The zero I_y is expected: the patch is symmetric top-to-bottom about the centre row.

Step 5 — Build the I_x matrix for the central 3×3 by applying S_x at each of the 9 pixels using the full 5×5 image:

- $(2,2)$: $(-10 + 20 - 20 + 40 - 30 + 50) = 50$
- $(2,3)$: $(-10 + 30 - 20 + 60 - 30 + 70) = 100$
- $(2,4)$: $(-20 + 30 - 40 + 60 - 50 + 70) = 50$
- $(3,2)$: $(-10 + 20 - 60 + 100 - 10 + 20) = 60$
- $(3,3)$: $(-10 + 30 - 60 + 140 - 10 + 30) = 120$
- $(3,4)$: $(-20 + 30 - 100 + 140 - 20 + 30) = 60$
- $(4,2)$: $(-30 + 50 - 20 + 40 - 10 + 20) = 50$
- $(4,3)$: $(-30 + 70 - 20 + 60 - 10 + 30) = 100$
- $(4,4)$: $(-50 + 70 - 40 + 60 - 20 + 30) = 50$

$$I_x = \begin{bmatrix} 50 & 100 & 50 \\ 60 & 120 & 60 \\ 50 & 100 & 50 \end{bmatrix}$$

Step 6 — Build the I_y matrix by applying S_y at each pixel:

- $(2,2)$: $(-10 - 20 - 20) + (30 + 60 + 10) = -50 + 100 = 90$
- $(2,3)$: $(-10 - 40 - 30) + (30 + 100 + 70) = -80 + 200 = 120$
- $(2,4)$: $(-20 - 60 - 30) + (50 + 140 + 70) = -110 + 260 = 150$
- $(3,2)$, $(3,3)$, $(3,4)$: $I_y = 0$ (rows above and below centre are mirror images)
- $(4,2)$: $(-30 - 60 - 50) + (10 + 20 + 20) = -140 + 50 = -90$
- $(4,3)$: $(-30 - 100 - 70) + (10 + 40 + 30) = -200 + 80 = -120$
- $(4,4)$: $(-50 - 140 - 70) + (20 + 60 + 30) = -260 + 110 = -150$

$$I_y = \begin{bmatrix} 90 & 120 & 150 \\ 0 & 0 & 0 \\ -90 & -120 & -150 \end{bmatrix}$$

3. Cornerness Score (3×3 neighbourhood) — Mean Normalised Step 1 — Calculate the means.

$$\mu_{I_x} = \frac{50 + 100 + 50 + 60 + 120 + 60 + 50 + 100 + 50}{9} = \frac{640}{9} \approx 71.11, \qquad \mu_{I_y} = 0$$

Step 2 — Normalise by subtracting the mean.

$$\widetilde{I}_x = I_x - \mu_{I_x} \approx \begin{bmatrix} -21.11 & 28.89 & -21.11 \\ -11.11 & 48.89 & -11.11 \\ -21.11 & 28.89 & -21.11 \end{bmatrix}, \qquad \widetilde{I}_y = I_y = \begin{bmatrix} 90 & 120 & 150 \\ 0 & 0 & 0 \\ -90 & -120 & -150 \end{bmatrix}$$

Step 3 — Compute sum of squared differences.

Using the variance formula $\sum \widetilde{I}^2 = \sum I^2 - 9\mu^2$:

$$\sum \widetilde{I}_x^2 = 51600 - 9\left(\tfrac{640}{9}\right)^2 = 51600 - \tfrac{409600}{9} = \tfrac{54800}{9} \approx 6088.89$$

$$\sum \widetilde{I}_y^2 = 90000 - 9(0)^2 = 90000$$

$$\sum \widetilde{I}_x \widetilde{I}_y = 0 \quad \text{(by symmetry of } \widetilde{I}_y\text{)}$$

Step 4 — Structure matrix.

$$\boxed{M = \begin{bmatrix} 6088.89 & 0 \\ 0 & 90000 \end{bmatrix}}$$

Step 5 — Eigenvalues (diagonal matrix $\Rightarrow$ eigenvalues on diagonal):

$$\lambda_1 = 90000, \qquad \lambda_2 \approx 6088.89$$

Both eigenvalues are significantly positive $\Rightarrow$ strong gradients in both directions $\Rightarrow$ this is a **corner**.

Step 6 — Harris score (with $k = 0.04$):

$$\begin{aligned} R &= \det(M) - k\left[\operatorname{tr}(M)\right]^2 \\ &= \left(\tfrac{54800}{9} \times 90000\right) - 0.04\,(6088.89 + 90000)^2 \\ &= 548{,}000{,}000 - 0.04 \times (96088.89)^2 \\ &\approx 548{,}000{,}000 - 369{,}322{,}963 \approx \mathbf{1.79 \times 10^8} \end{aligned}$$

$R \gg 0$ confirms a strong **corner**.

1. Bilateral Filter — Detailed Construction

Question 4: Hough Transform for Ellipses

An ellipse whose axes are parallel to the coordinate axes is described by:

$$\left(\frac{x - c_x}{r_x}\right)^2 + \left(\frac{y - c_y}{r_y}\right)^2 = 1 \tag{1}$$

where (c_x, c_y) is the ellipse's centre, and r_x, r_y are its horizontal and vertical radii (both **known**).

A. What is the dimensionality of the Hough space?

B. What is the shape in Hough space corresponding to a point in image space?

C. Write pseudocode for a basic Hough transform procedure for ellipse detection.

D. Taking the derivative wrt x: $\left(\frac{2}{r_x^2}\right)(x - c_x) + \left(\frac{2}{r_y^2}\right)(y - c_y)\frac{dy}{dx} = 0$. Redesign your procedure to use gradient information. Explain how A and B change. Also present an alternative approach that reduces the Hough space dimensionality.

Answers

Part 1: Basic Hough Transform (No Gradient)

a. The dimensionality is **2D**. There are two unknowns: the center coordinates c_x and c_y.

b. Re-arranging the equation: $\left(\frac{c_x - x}{r_x}\right)^2 + \left(\frac{c_y - y}{r_y}\right)^2 = 1$.
For a fixed image point (x, y), this corresponds to an **ellipse** in the Hough space centered at (x, y) with radii r_x and r_y.

c. Algorithm:

1. Detect edge points in the image.
2. Initialize accumulator array $H(c_x, c_y)$ to all zeros.
3. For each edge point (x, y):
   - For all possible values of c_x:
   - Compute $c_y = y \pm r_y \sqrt{1 - \left(\frac{c_x - x}{r_x}\right)^2}$
   - Increment $H(c_x, c_y) + +$
4. Find local maxima in $H(c_x, c_y)$ to identify centers.

Part 2: Using Gradient Information

d. By using the gradient $\frac{dy}{dx}$, we now have two equations: the ellipse equation and its derivative.

A. The dimensionality remains **2D** (the parameters are still c_x, c_y), but the mapping efficiency changes.

B. Each point in image space now maps to a **single point** (or a small set of points) in Hough space rather than a full ellipse.

C. Algorithm Update: For each edge point, we solve the two equations to obtain c_x, c_y directly. This moves the complexity from the accumulation loop (1D search) to a direct calculation.

Question 5: Hough Transform for Spheres

We have a 3D point cloud and wish to find all 3D spheres using the Hough transform. Ranges: $x_{\max} = 10$, $y_{\max} = 20$, $z_{\max} = 30$.

1. Write the equation of the sphere.
2. Determine the parameter space dimensionality.
3. What is the count of voxels $(1 \times 1 \times 1)$ in the accumulator if the radius is known?
4. Write pseudocode to find all 3D spheres with an unknown radius.

Answers

1. Sphere Equation

$$(x - c_x)^2 + (y - c_y)^2 + (z - c_z)^2 = r^2$$

2. Parameter Space Dimensionality Parameters are $(c_x, c_y, c_z, r) \Rightarrow$ **4D**. If r is known $\Rightarrow$ **3D**.

3. Number of Voxels (known radius)

$$10 \times 20 \times 30 = \mathbf{6{,}000} \text{ voxels}$$

4. Pseudocode (Unknown Radius)

```
Initialize A[cx][cy][cz][r] = 0 for all cx, cy, cz, r

For each point (x0, y0, z0) in the point cloud:
    If it is an edge point:
        For cx in range(0, 10):
            For cy in range(0, 20):
                For cz in range(0, 30):
                    r = sqrt((x0-cx)^2 + (y0-cy)^2 + (z0-cz)^2)
                    A[cx][cy][cz][r]++

Find (cx, cy, cz, r) where A is a local maximum
Reconstruct the sphere using the found parameters
```

Question 6: Gaussian Kernel Convolution Property

Statement: Applying the Gaussian kernel with bandwidth $\sigma = 1$ twice on an image is equivalent to applying another Gaussian kernel with bandwidth $\sigma = \sqrt{2}$. Prove.

Proof

Drop the normalisation constant and write the Gaussian as:

$$G = e^{-\dfrac{x^2 + y^2}{2\sigma^2}} \qquad \xrightarrow{\text{Fourier}} \qquad \hat{G} = e^{-\frac{\sigma^2(u^2+v^2)}{2}}$$

For $\sigma = 1$:

$$G = e^{-\dfrac{x^2 + y^2}{2}}$$

Applying the filter twice corresponds to convolution $G * G$. In the Fourier domain, convolution becomes multiplication:

$$G * G \xrightarrow{\text{Fourier}} \hat{G} \cdot \hat{G} = e^{-\dfrac{u^2 + v^2}{2}} \cdot e^{-\dfrac{u^2 + v^2}{2}} = e^{-(u^2+v^2)}$$

This result must match a Gaussian with bandwidth σ_F:

$$\hat{G}_F = e^{-\dfrac{(u^2+v^2)}{2\sigma_F^2}}$$

Equating exponents:

$$\frac{1}{2\sigma_F^2}(u^2+v^2) = (u^2+v^2) \implies \sigma_F^2 = \frac{1}{2} \implies \sigma_F = \frac{1}{\sqrt{2}} \quad \text{(in Fourier domain)}$$

Returning from the Fourier domain, the equivalent spatial-domain bandwidth is:

$$\boxed{\sigma = \sqrt{2}}$$

Hence applying a $\sigma = 1$ Gaussian twice is equivalent to a single $\sigma = \sqrt{2}$ Gaussian. ■

Question 7: Inverse Rank & Gaussian Kernels

The inverse rank function can approximate the Gaussian function

$$\frac{1}{\sqrt{2\pi\sigma^2}}\, e^{-\frac{x^2}{2\sigma^2}}.$$

Using the (1/rank) of the nearest distance from the anchor:

A. What is the 1×5 kernel using the Gaussian kernel?

B. What is the 1×5 kernel using the inverse rank kernel?

C. What are the 2D 5×5 kernels using the inverse rank function? Assume separability can be applied, then apply the filters on the centre pixel of image I.

D. Suggest an approach to approximate the 5×5 Laplacian kernel using the inverse rank kernel.

Given image:

$$I = \begin{bmatrix} 10 & 10 & 40 & 50 & 50 \\ 10 & 10 & 40 & 50 & 50 \\ 50 & 50 & \boxed{50} & 10 & 10 \\ 10 & 10 & 40 & 50 & 50 \\ 10 & 10 & 40 & 50 & 50 \end{bmatrix}$$

Answers

A. 1×5 Gaussian Kernel Using $\sigma = 1$ and evaluating $e^{-x^2/(2\sigma^2)}$ at $x \in \{-2, -1, 0, 1, 2\}$:

$$\mathbf{g} \propto \left[e^{-2},\ e^{-0.5},\ 1,\ e^{-0.5},\ e^{-2}\right] \approx [0.135,\ 0.607,\ 1,\ 0.607,\ 0.135]$$

Normalising (sum = 2.484):

$$\boxed{G_{1\times5} \approx [0.054,\ 0.244,\ 0.403,\ 0.244,\ 0.054]}$$

B. 1×5 Inverse Rank Kernel Distances from the centre anchor: $d = [2, 1, 0, 1, 2]$. Rank the distances (smallest = 1): rank $= [5, 3, 1, 3, 5]$ (ties broken symmetrically).

Inverse ranks: $[1/5,\ 1/3,\ 1,\ 1/3,\ 1/5]$. Sum $= 1/5+1/3+1+1/3+1/5 = \frac{3+5+15+5+3}{15} = \frac{31}{15}$.

Normalised inverse rank kernel:

$$\boxed{K_{1\times5} = \frac{15}{31}\left[\tfrac{1}{5},\ \tfrac{1}{3},\ 1,\ \tfrac{1}{3},\ \tfrac{1}{5}\right] \approx [0.097,\ 0.161,\ 0.484,\ 0.161,\ 0.097]}$$

C. 2D 5×5 Inverse Rank Kernel (Separable) and Application **Step 1 — Form the 2D kernel** by the outer product $K_{5\times5} = K_{1\times5}^{\top} K_{1\times5}$ (using unnormalised weights $w = [1/5,\ 1/3,\ 1,\ 1/3,\ 1/5]$):

$$K_{5\times5}^{\text{unnorm}} = \begin{bmatrix} \frac{1}{25} & \frac{1}{15} & \frac{1}{5} & \frac{1}{15} & \frac{1}{25} \\ \frac{1}{15} & \frac{1}{9} & \frac{1}{3} & \frac{1}{9} & \frac{1}{15} \\ \frac{1}{5} & \frac{1}{3} & 1 & \frac{1}{3} & \frac{1}{5} \\ \frac{1}{15} & \frac{1}{9} & \frac{1}{3} & \frac{1}{9} & \frac{1}{15} \\ \frac{1}{25} & \frac{1}{15} & \frac{1}{5} & \frac{1}{15} & \frac{1}{25} \end{bmatrix}$$

Step 2 — Apply separably to the centre pixel of I (row 3, col 3).

Pass 1 — horizontal 1D filter on each row of the 5×5 patch:
For each row r, compute $h_r = K_{1\times5}\cdot\text{row}_r$, using unnormalised weights $w = [1/5, 1/3, 1, 1/3, 1/5]$ and then normalise at the end.

$$\text{row}_3 = [50, 50, 50, 10, 10]$$

$$\begin{aligned}
h_1 &= \tfrac{1}{5}(10) + \tfrac{1}{3}(10) + 1(40) + \tfrac{1}{3}(50) + \tfrac{1}{5}(50) = 2 + 3.33 + 40 + 16.67 + 10 = 72 \\
h_2 &= \tfrac{1}{5}(10) + \tfrac{1}{3}(10) + 1(40) + \tfrac{1}{3}(50) + \tfrac{1}{5}(50) = 72 \\
h_3 &= \tfrac{1}{5}(50) + \tfrac{1}{3}(50) + 1(50) + \tfrac{1}{3}(10) + \tfrac{1}{5}(10) = 10 + 16.67 + 50 + 3.33 + 2 = 82 \\
h_4 &= 72 \quad \text{(same as row 1 by symmetry)} \\
h_5 &= 72
\end{aligned}$$

Pass 2 — vertical 1D filter on the column of horizontal results:

$$\begin{aligned}
\text{output} &= \tfrac{1}{5}h_1 + \tfrac{1}{3}h_2 + 1\cdot h_3 + \tfrac{1}{3}h_4 + \tfrac{1}{5}h_5 \\
&= \tfrac{1}{5}(72) + \tfrac{1}{3}(72) + 82 + \tfrac{1}{3}(72) + \tfrac{1}{5}(72) \\
&= 14.4 + 24 + 82 + 24 + 14.4 = 158.8
\end{aligned}$$

Normalisation factor $W = (1/5 + 1/3 + 1 + 1/3 + 1/5)^2 = (31/15)^2 = 961/225$.

$$\boxed{\tilde{I}(3,3) = \frac{158.8}{961/225} = \frac{158.8 \times 225}{961} \approx 37.2}$$

D. Approximating the 5×5 Laplacian Kernel The Laplacian of Gaussian (LoG) can be approximated as a *Difference of Gaussians* (DoG), and equivalently using the inverse rank kernel as:

$$\boxed{L \approx \delta - G_{\text{inv}}}$$

where δ is the identity (impulse kernel: 1 at centre, 0 elsewhere) and G_{inv} is the normalised 5×5 inverse rank kernel constructed in part C.

Explicitly:

$$L_{5\times5} = I_{5\times5} - K_{5\times5}^{\text{norm}}$$

Question 8: Gaussian Pyramid & Laplacian Pyramid

We compute the Gaussian pyramid for a 4096×4096 grayscale image. Each pixel is stored using one byte.

A. Determine the total size (in bytes) of the Gaussian pyramid if the smallest resolution is 4×4.

B. What is the size of the corresponding Laplacian pyramid if you know that the residue values of each pixel range only from -32 to 31?

Answers

A. Total Size of the Gaussian Pyramid Starting at 4096×4096 and halving until 4×4, the levels are:

$$4096^2, \quad 2048^2, \quad 1024^2, \quad 512^2, \quad 256^2, \quad 128^2, \quad 64^2, \quad 32^2, \quad 16^2, \quad 8^2, \quad 4^2$$

That is 11 levels. The total number of pixels (= bytes, since 1 byte/pixel) is:

$$\begin{aligned} S &= 4096^2 + 2048^2 + 1024^2 + \cdots + 4^2 \\ &= 4096^2 \left(1 + \frac{1}{4} + \frac{1}{16} + \cdots + \frac{1}{4^{10}}\right) \\ &\approx 22{,}369{,}621 \text{ bytes} \end{aligned}$$

$$\boxed{S \approx 22{,}369{,}621 \text{ bytes} \approx 21.33 \text{ MB}}$$

B. Size of the Laplacian Pyramid The Laplacian pyramid stores residuals (differences between Gaussian levels) at every level *except* the coarsest, plus the coarsest Gaussian level itself.

Bits per residual pixel:
The residue values range from -32 to 31, a total of $64 = 2^6$ values. Therefore each residual pixel requires **6 bits**.

Storage breakdown:

- 10 residual levels (from 4096^2 down to 8^2): each pixel stored in 6 bits.

- 1 coarsest level ($4^2 = 16$ pixels): stored at full 8 bits (1 byte each).

Total bits for residual levels:

$$\begin{aligned} B_{\text{res}} &= 6 \times \left(4096^2 + 2048^2 + \cdots + 8^2\right) \\ &= 6 \times 4096^2 \left(1 + \tfrac{1}{4} + \cdots + \tfrac{1}{4^9}\right) \\ &\approx 6 \times 22{,}369{,}605 \approx 134{,}217{,}630 \text{ bits} \end{aligned}$$

Converting residual levels to bytes:

$$\frac{134{,}217{,}630}{8} \approx 16{,}777{,}203 \text{ bytes}$$

Adding the coarsest level (16 pixels $\times$ 1 byte):

$$S_L = 16{,}777{,}203 + 16 \approx 16{,}777{,}219 \text{ bytes}$$

$$\boxed{S_L \approx 16{,}777{,}219 \text{ bytes} \approx 16 \text{ MB}}$$

This is a **saving of** ≈ 5.33 MB compared to the Gaussian pyramid, because the residuals require only 6 bits instead of 8 bits per pixel.

For more exam problems and resources, visit:
More Computer Vision Questions